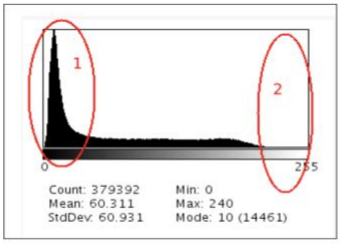
Report

Partie 1 : Types d'images et statistiques images (2 pts)

Q1 Ouvrez l'image monalisa.png et convertissez-la en niveaux de gris. A l'aide d'openCV, générez et afficher l'histogramme de niveaux de gris, et expliquez la répartition des niveaux de gris dans l'histogramme aux points 1 et 2.
https://docs.opencv.org/3.4.2/d1/db7/tutorial_py_histogram_begins.html



In this task I load the Mona Lisa image in grayscale and compute its histogram using `cv2.calcHist`. A histogram shows how pixel intensities are distributed. In grayscale we only need one channel. We use 256 bins from 0 to 255. Histograms are useful for thresholding, segmentation and contrast enhancement through equalization.

Q2 Ouvrez et convertissez en niveaux de gris, les images paysage et soleil. Pour chacune d'entre-elles

- Générez l'histogramme de niveaux de gris
- Egalisez l'histogramme
- Générez l'histogramme de niveaux de gris après égalisation

Lorsque nécessaire, insérez dans votre rapport l'histogramme et l'image avant et après égalisation **et surtout expliquez**

In this task I apply histogram equalization to improve image contrast. First we compute the histogram of the original grayscale image. Then we use `cv2.equalizeHist` to remap pixel intensities based on the cumulative distribution function The result is an equalized image with improved contrast. This is confirmed by the histogram: before intensities are clustered and aftert they are spread out.

Partie 2 : Filtrage, convolution, détection de contours (4 pts)

Q1: Ouvrez à nouveau monalisa.png (ou autre image de votre choix) et convertissez-les en niveaux de gris. Appliquez une seule fois des filtres moyenneurs de taille/rayon variable :

1. 5X5 2. 9X9 3. 15X15

Expliquez ? Que donnerait l'application d'un filtre moyenneur dont la dimension serait égale à celle de l'image ?
https://docs.opencv.org/3.4.2/d2/d96/tutorial_py_table_of_contents_imgproc.html

Q2: Appliquer d'autres traitements de filtrage passe bas (flou gaussien, masque median, …). Insérez certains résultats dans votre rapport et **commentez-les afin de démontrer que vous avez compris** le mécanisme de filtrage/convolution d'images

Q3: Effectuez les actions suivantes et reportez dans votre rapport les coefficients de filtres utilisés et les images filtrées obtenues :

- Ouvrez l'image zebre.jpg et convoluez la avec un filtre permettant de faire ressortir les traits horizontaux
- Ouvrez l'image suzan.jpg et convoluez la avec un filtre permettant de faire ressortir les traits verticaux

Expliquez les masques utilisés

Q4: Appliquer les algorithmes de détection de contours Laplacien et Canny. Insérez certains résultats dans votre rapport et commentez-les afin de démontrer que vous avez compris le mécanisme de filtrage passe-haut d'images.

Q5: Selon vous, quels sont les points faibles de toutes ces méthodes de filtrage par convolution ?

In this task I apply mean filtering with kernel sizes 5×5, 9×9, and 15×15. The mean filter replaces each pixel with the average of its neighbors. That reduces noise but also blurs edges and details. Increasing kernel size makes the image smoother but more blurred.

Gaussian blur: Smooths the image and reduces noise, but preserves edges better than a mean filter. Median filter: Removes isolated noise (salt-and-pepper) very effectively, while preserving edges. Both are low-pass filters but their effect on details and noise is different.

Laplacian is a high-pass filter that detects edges by highlighting regions of rapid intensity change (second derivative). Canny is a more advanced edge detector that uses gradients non-maximum suppression and hysteresis thresholding. Both are high-pass filters but Canny is less sensitive to noise and produces cleaner edge maps.

Partie 3 : Images binaires et opérations entre images (5 pts)

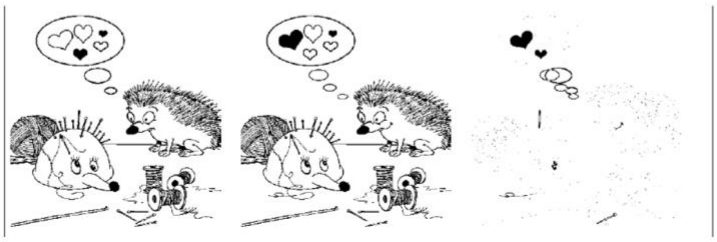
Q1: Complétez le code fourni afin d'appliquer différents algorithmes de seuillage / binarisation (seuillage fixe ou adaptatif) à différentes images fournies. Insérez certains résultats dans votre rapport et commentez-les afin de **démontrer que vous avez compris le mécanisme de binarisation**

https://docs.opencv.org/3.4.2/d2/d96/tutorial_py_table_of_contents_imgproc.html

Q2: Ouvrez les images jeu1 et jeu2.

A l'aide d'opérations arithmétiques entre ces deux images, mettez (comme sous l'image ci-dessous) en évidence les 7 différences existant entre ces deux images.

Proposez un algorithme (sans forcément l'implémenter) capable de résoudre ce problème lorsque les images à comparer sont susceptibles d'être décalées de quelques pixels l'une par rapport à l'autre (CF jeu3).

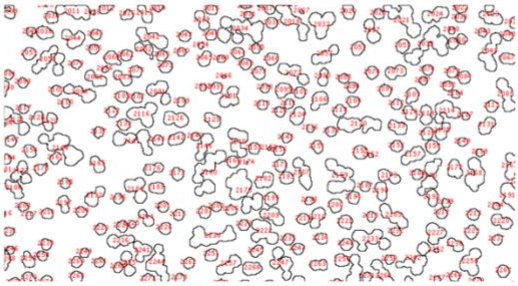


The 7 differences between the two images:

1. Number of hearts in the hedgehog's thought bubble.
2. Number of bubbles in the hedgehog's thought bubble.
3. Shape of the spool of thread.
4. Appearance of the thread on the spool.
5. Shape of the hedgehog's nose.
6. Position of eye of the little hedgehog.
7. Position/count of needles on the ground.

I carefully compared each part of the two images looking for objects or details that were changed, added or removed. This is similar to what an image processing algorithm would do: it would subtract one image from the other and highlight the areas where there are differences.

Q3: Des biologistes aimeraient compter le nombre de cellules présentes dans des images. Proposez un algorithme permettant de compter les cellules automatiquement. Vérifiez la présence de 329 cellules dans l'image cellule.png.
https://docs.opencv.org/3.4/d3/dc0/group__imgproc__shape.html#ga107a78bf7cd25dec05fb4dfc5c9e765f



In this task I count cells in a microscope image. I first apply Otsu's thresholding to binarize the grayscale image, automatically separating cells from the background. Then we find all contours of white regions and filter them by area to remove noise. The number of remaining contours gives us the detected cell count. This method is simple and effective for separated cells

Q4 : Malheureusement, une vérification faite manuellement par un biologiste indique que l'algorithme s'est trompé et que le nombre réel de cellules avoisine plutôt les 370.
Une rapide analyse de l'image vous permet de trouver l'origine de cette erreur. En effet, une mauvaise utilisation du système d'acquisition a généré deux types de bruits artificiels :
1. Des petites taches noires sont apparues un peu partout et sont interprétées comme étant des cellules (cf cas 1 de l'image ci-dessous)
2. Des cellules sont collées sur la photo alors qu'elles sont dissociées dans la réalité (cf cas 2 de l'image ci-dessous)



In this task I detect cells in a microscope image using Otsu's thresholding and contour analysis. First we binarize the image, then find all contours of white regions. We filter out small areas to remove noise and keep only real cells. Each valid contour is drawn in red, and we compute its centroid using image moments to label it. The final result shows the number of detected cells and their positions. This approach works well for objects with distinct intensity compared to the background.

Your work

1. Select 2 objects in your surroundings.
2. Take 5 pictures for each object.
   > change angle, rotation, scale, luminosity
3. Perform SIFT matching for each pair of image of the same object.
4. Display results and comment.

In this task I use SIFT to detect and match features between multiple images of the same object. SIFT is scale and rotation invariant. Each image is converted to grayscale and processed with `detectAndCompute` to extract keypoints and descriptors. I use a Brute-Force matcher to compare descriptors between two images. Lowe's ratio test keeps only distinctive matches. Finally, we visualize the matches with `cv2.drawMatches`. A higher number of good matches means the two images are of the same object.

Your work

- Given an image input : solve the captcha!
- Output : bounding box of the area where to click to select the correct answer.

- Write pseudo-algo
- Implement
- Qualitative tests
- Quantitative tests

My algorithm solves the CAPTCHA by first preprocessing the image with grayscale conversion and Otsu thresholding to get binary icons. I extract each icon using contours, crop and resize them to a standard 32×32 size. Then I compare every icon against the others under all four rotations to achieve rotation invariance. For each icon I count how many times it appears, and finally I select the least frequent icon as the CAPTCHA solution. The complexity is quadratic in the number of icons but since there are few icons it's efficient.

Pseudo algorithm:

```
FUNCTION SolveCaptcha(image):

    # Step 1: Preprocessing
    Load image and convert to grayscale
    If background is light:
        Invert grayscale
    Apply Otsu threshold to get binary mask

    # Step 2: Extract icons
    Find contours in binary image
    For each contour:
        Compute bounding box
        If box size is valid:
            Crop icon
            Resize to fixed size
            Store (bounding box, resized image)

    # Step 3: Count occurrences
    For each icon:
        Initialize count = 0
        For each other icon:
            Compare using similarity under 0°, 90°, 180°, 270° rotations
            If similarity > threshold:
                Increase count
        Save count

    # Step 4: Find rarest icon
    Select icon with minimum count
    Draw rectangle around it
    Return its bounding box
```